

AFRILANG Stdlib

std/c/comprunlen

Français. Bibliothèque AFRILANG 'std/c/comprunlen' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : runlenEncode, runlenDecode, runlenRatio, runlenSize, runlenCompressed, runlenRoundTrip.

```
'import "std/c/comprunlen"' · 'use comprunlen'  
- 'runlenEncode(data list number) → list number' - 'runlenDe-  
code(encoded list number) → list number' - 'runlenRatio(data list num-  
ber) → number' - 'runlenSize(data list number) → number' - 'runlen-  
Compressed(data list number) → number' - 'runlenRoundTrip(data list  
number) → bool'
```

English. AFRILANG library 'std/c/comprunlen' (tier complex, category complex). Exports 6 function(s): runlenEncode, runlenDecode, runlenRatio, runlenSize, runlenCompressed, runlenRoundTrip.

```
'import "std/c/comprunlen"' · 'use comprunlen'  
- 'runlenEncode(data list number) → list number' - 'runlenDe-  
code(encoded list number) → list number' - 'runlenRatio(data list num-  
ber) → number' - 'runlenSize(data list number) → number' - 'runlen-  
Compressed(data list number) → number' - 'runlenRoundTrip(data list  
number) → bool'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/comprunlen' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : runlenEncode, runlenDecode, runlenRatio, runlenSize, runlenCompressed, runlenRoundTrip.

```
'import "std/c/comprunlen"' · 'use comprunlen'
```

```
- `runlenEncode(data list number) → list number`
- `runlenDecode(encoded list number) → list number`
- `runlenRatio(data list number) → number`
- `runlenSize(data list number) → number`
- `runlenCompressed(data list number) → number`
- `runlenRoundTrip(data list number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/comprunlen"
use comprunlen
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- runlenEncode(data list number) → list•
runlenDecode(encoded list number) → list•
runlenRatio(data list number) → number•
runlenSize(data list number) → number•
runlenCompressed(data list number) → number•
runlenRoundTrip(data list number) → bool

4. Exemples d'utilisation

```
import "std/c/comprunlen"
use comprunlen
```

```
create result = runlenEncode(/* args */)
say result
```

```
create result = runlenDecode(/* args */)
say result
```

```
create result = runlenRatio(/* args */)
say result
```

```
create result = runlenSize(/* args */)
say result
```

```
create result = runlenCompressed(/* args */)
say result
```

```
create result = runlenRoundTrip(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/comprunlen"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module comprunlen

```
function runlenEncode(data list number) returns list number  
end
```

```
function runlenDecode(encoded list number) returns list number  
end
```

```
function runlenRatio(data list number) returns number  
end
```

```
function runlenSize(data list number) returns number  
end
```

```
function runlenCompressed(data list number) returns number  
end
```

```
function runlenRoundTrip(data list number) returns bool  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/comprunlen' (tier complex, category complex). Exports 6 function(s): runlenEncode, runlenDecode, runlenRatio, runlenSize, runlenCompressed, runlenRoundTrip.

'import "std/c/comprunlen"' · 'use comprunlen'

```
- `runlenEncode(data list number) → list number`
- `runlenDecode(encoded list number) → list number`
- `runlenRatio(data list number) → number`
- `runlenSize(data list number) → number`
- `runlenCompressed(data list number) → number`
- `runlenRoundTrip(data list number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/comprunlen"
use comprunlen
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- runlenEncode(data list number) → list•
runlenDecode(encoded list number) → list•
runlenRatio(data list number) → number•
runlenSize(data list number) → number•
runlenCompressed(data list number) → number•
runlenRoundTrip(data list number) → bool

4. Usage examples

```
import "std/c/comprunlen"
use comprunlen
```

```
create result = runlenEncode(/* args */)
say result
```

```
create result = runlenDecode(/* args */)
say result
```

```
create result = runlenRatio(/* args */)
say result
```

```
create result = runlenSize(/* args */)
say result
```

```
create result = runlenCompressed(/* args */)
say result
```

```
create result = runlenRoundTrip(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/comprunlen"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module comprunlen

```
function runlenEncode(data list number) returns list number
end
```

```
function runlenDecode(encoded list number) returns list number
end
```

```
function runlenRatio(data list number) returns number
end
```

```
function runlenSize(data list number) returns number
end
```

```
function runlenCompressed(data list number) returns number
end
```

```
function runlenRoundTrip(data list number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/comprunlen"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/comprunlen/>

Source (extrait)

```
module comprunlen

function runlenEncode(data list number) returns list number
end

function runlenDecode(encoded list number) returns list number
end

function runlenRatio(data list number) returns number
end

function runlenSize(data list number) returns number
end

function runlenCompressed(data list number) returns number
```

```
end
function runlenRoundTrip(data list number) returns bool
end
end
```